EE2016 Microprocessors Theory and Lab, Aug - Nov 2025

Instructor: Dr. R. Manivasakan. Tutorial 1

EE Dept, IIT Madras.

(SOC) Embedded Processor → integrates RAM, ROM, I/O ports, interrupt controllers onto a single chip, making it power-efficient

1 Fill in the blanks

1. Intel's embedded processor "embeds" the memory controller hub (I/O controller hub) into the microprocessor.
2. One can fabricate microcontroller (microcontroller / microprocessor) by incorporating memory & peripherals modules to microprocessor (microcontroller / microprocessor).
3. If the computational resources: ~~super computer~~, ~~wearable~~, ~~main frame~~, ~~desktop~~, ~~server~~, ~~IoT~~, ~~workstation~~ are arranged in the ascending order of their computational power, then the order would be wearable < IoT (Internet of Things) < desktop < workstation < server < mainframe < super computer
4. The harware realization of multiplier in a microcontroller / microprocessor is a architectural (architectural / organizational) issue. ↳ how the architectural features are physically implemented
5. Data bus most of the time is bidirectional (unidirectional / bidirectional) while the address bus is always unidirectional (unidirectional / bidirectional). ↳ transfer data, CPU reads data from memory I/P and writes data to memory O/P; used by µP to specify a memory location (or) I/O device (CPU tells where to go)
6. Major difference between the microcontroller and the microprocessor is microcontroller → self contained computer on a single chip; microprocessor → only the CPU
7. In the von Neumann architecture discussed in the class, the MBR (designed for specific data) handles a word of length 40 (20 or 40 bits). Similarly, IR 8 bits IBR 20 bits MAR 12 bits and finally PC 12 bits. (temporarily holds 2nd instruction from a fetched 40-bit word while 1st is being executed; memory words were 40 bits long; instruction to data are stored in same memory space, CPU can't fetch instruction & data @ same time)
8. Output of IR is a control (address / control / data) signal. (IR → holds the opcode of instruction being executed; instruction register (holds to opcode); memory address register upto 4096 memory locations; Program counter (address of next instruction to be fetched))
9. Issues or performance measures of whether a hardware multiplier implementation or software realization of multiplier algorithm in a microprocessor design, are anticipated freq. of use, speed, cost, and physical size (word → fundamental unit of data that a processor can handle at one time (32 / 64 bits generally); 1024 words ⇒ 10 bits are required to identify a memory word)
10. Given a memory word of 40 bits (corresponding to a row) and 1024 such rows are there, (a) how many bits are required to uniquely identify a memory word? (b) size of memory in bits? 10; 40 × 1024 = 40960 bits memory

2 Answer the following

1. Browse in the internet for a commercial processor which has also FPGA "embedded" in it. Intel Agilex, Intel Stratix
2. What is the difference between computer organization and architecture? how? ← organization → how the architectural attribute/instruction is realized internally using hardware. what? ← architecture → attributes of computing system which are explicit during logical execution of the program
3. What is a program counter? What does it count? PC is a special purpose register inside CPU. keeps track of sequence of instructions that are being executed. It holds the memory address of next instruction to be fetched and executed.
4. What is the difference between system bus and internal bus in a microprocessor?
5. Why is that the when the CPU frequency increases, heat dissipated also increases? when CPU freq ↑, reactive current flows through a path corresponding to capacitor. This current in real world ↑ resistive losses producing heat; transistors switch states more often as frequency increases. Since each switch consumes a small amount of energy, that is released as heat. ($P = CV^2f$ (FT ⇒ DT))
6. Describe the Von Neumann architecture with a block diagram and explain its operation. Indicate the bus size in each of the buses out there.
7. What is the major difference between Von Neumann architecture and Harvard architecture? Von Neumann uses a single memory & single bus for both data and instructions, while Harvard uses seperate memories and seperate busses for data and instructions. In Von-Neumann instructions and data can't be accessed at the same time, and the design is simple and cheap

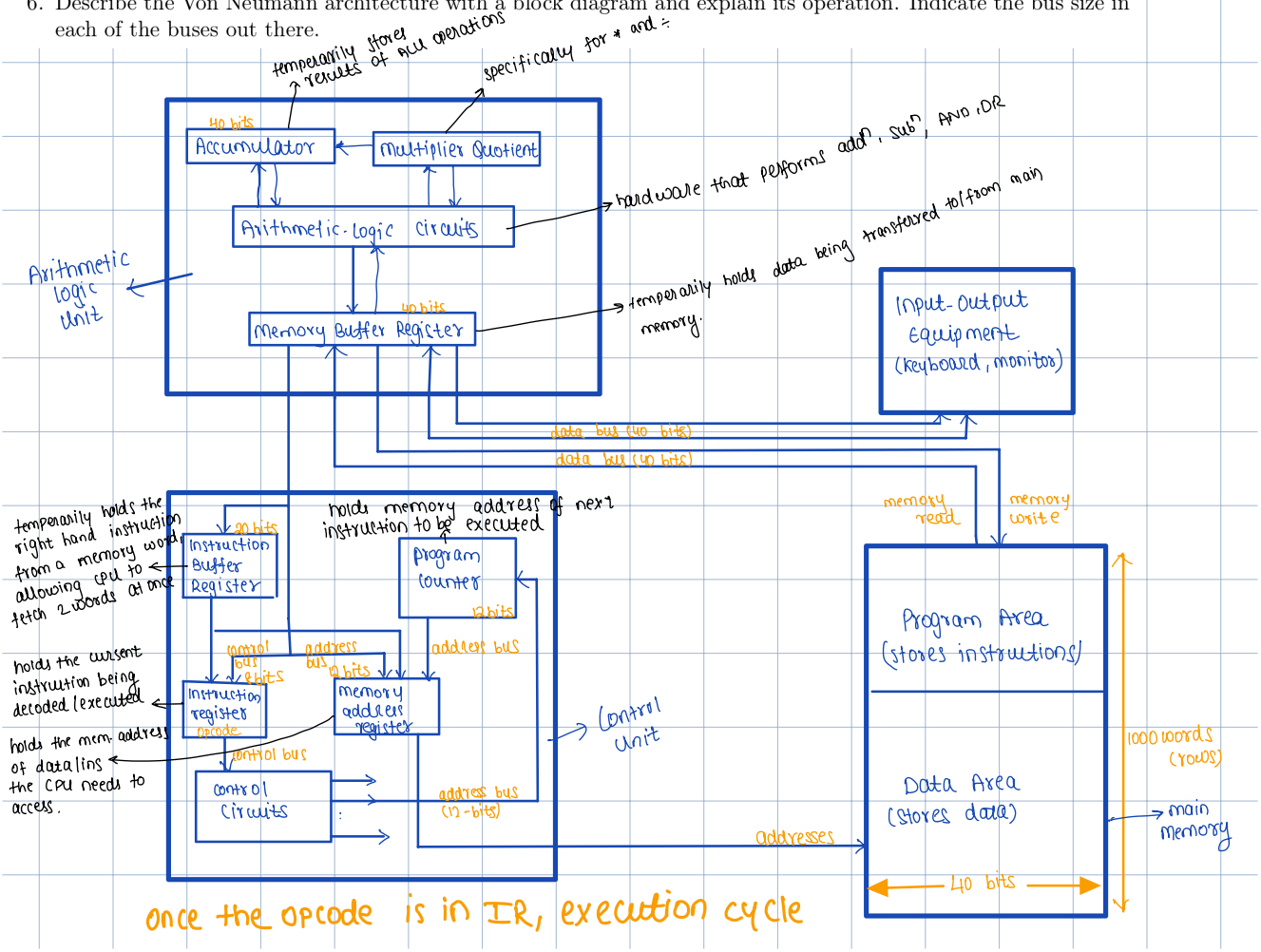
System bus

- → Outside the µP, on the motherboard
- → connects CPU with to other major components like RAM, storage controllers, I/O devices
- → slower than Internal bus
- → 3 parts:
  - Address bus: carried memory addresses (uni)
  - control bus: carries control signals (mix)
  - data bus: carries actual data (bi)

internal bus

- → inside the µP
- → connects internal components of CPU such as ALU, CU, registers
- → Faster than System bus
- → mostly bidirectional

6. Describe the Von Neumann architecture with a block diagram and explain its operation. Indicate the bus size in each of the buses out there.

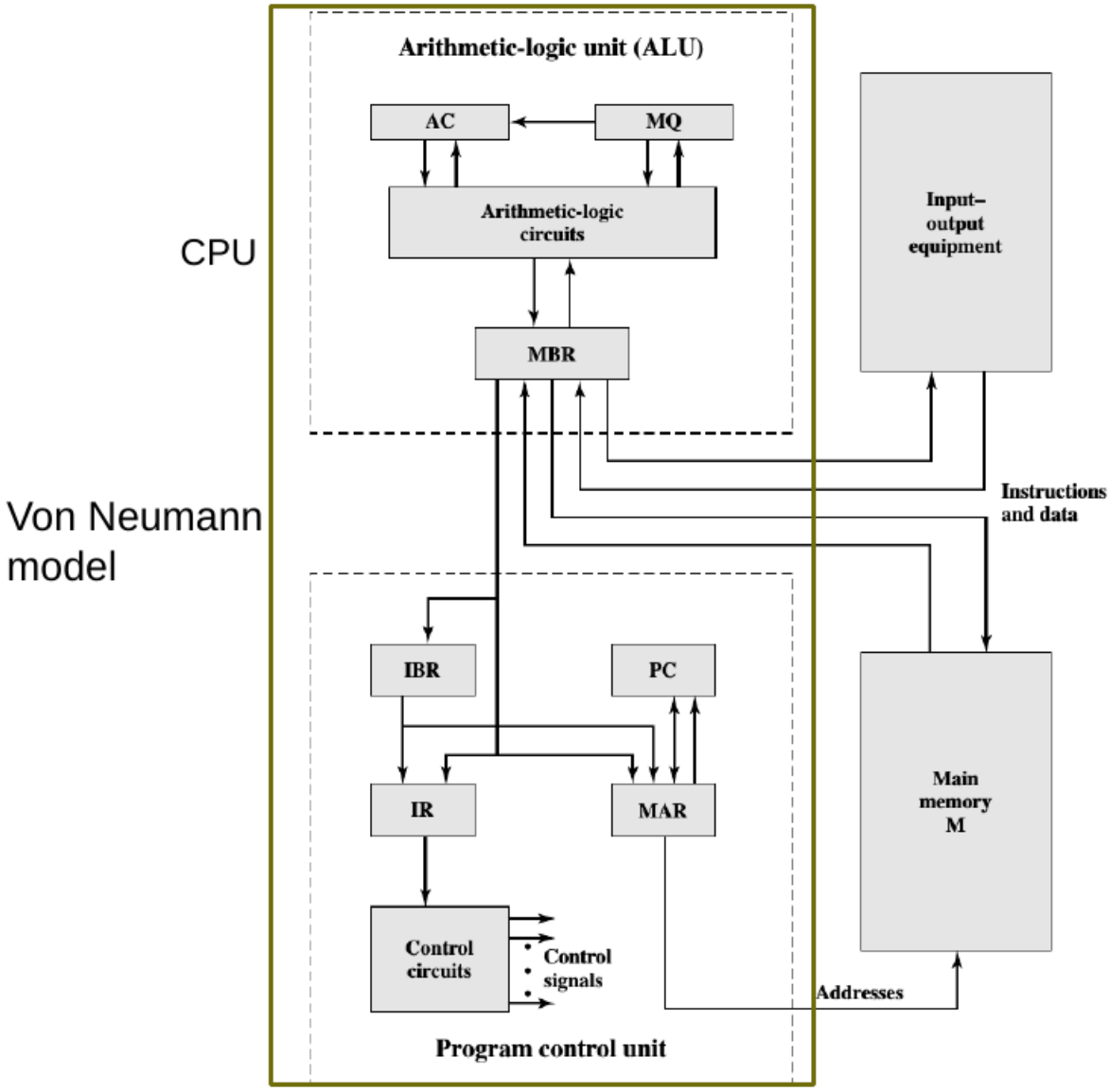


once the opcode is in IR, execution cycle

begins

3 Solve the Following

1. Recall the van Neumann computer hardware architecture, which is given before for convenience.



Answer the following

(a) What is the type of bus which connects MBR from main memory? What is the size of the bus? (Meaning, how many bits does the bus possess?). What are two main types of data it can carry?
data bus 40 bits

(b) Repeat (a) with direction changed. data bus, 40 bits, data instruction, Data

(c) What is fed to IR block? How many bits does it have? op-code ; 8-bits

(d) Can MAR has something to send to PC? On the other hand, what does PC send to MAR? What is the type of bus? And its size?
No PC sends the addresses of instructions to be executed
address bus 12 bits

(e) What type of bus is used to connect MAR to main memory? What is its size?
address bus 12-bits

(f) Given that the number of rows in the main memory is 1000, and each row contains 40 bits, what is the size of memory in bytes? 40x1000/8 = 5000 bytes

(g) Given that all the instructions are of constant length (RISC feature), of 20 bits, how many instructions are accommodated in a row in the main memory (in the program area)? 40/20 = 2 instructions

(h) Consider the algorithmic part - flow chart - of van Neumann model. Where is that the IBR is declared 'empty', so that the condition testing block (at the top) can use that information? Indicate it in the flow chart.

2. Recall the von Neumann computer introduced in the class. The following Fig 1 gives the execution of instructions, in which MAR & MBR are implicit. In the following Figure, one needs to include a nibble 0 in the MSB of word corresponding to AC and IR. While the address bits are 12, the word stored in memory is also increased by 4 bits leading to 20 bit memory word. While the accumulator AC (having 20 bits in length), corresponds to MBR, the 20 bit word below it, in each step, corresponds to IR (MSB 8 bits) and MAR (LSB 12 bits). For example read, "|1|9|4|0| IR" as "|0|1|9|4|0| IR MAR" wherein the LSB 12 bits (0x 940) are MAR contents and MSB 8 bits (0x 01) are IR contents. Only a generic instruction (it can be left or right instruction) is shown. AC length is also 20 bits (or 5 nibbles in length).

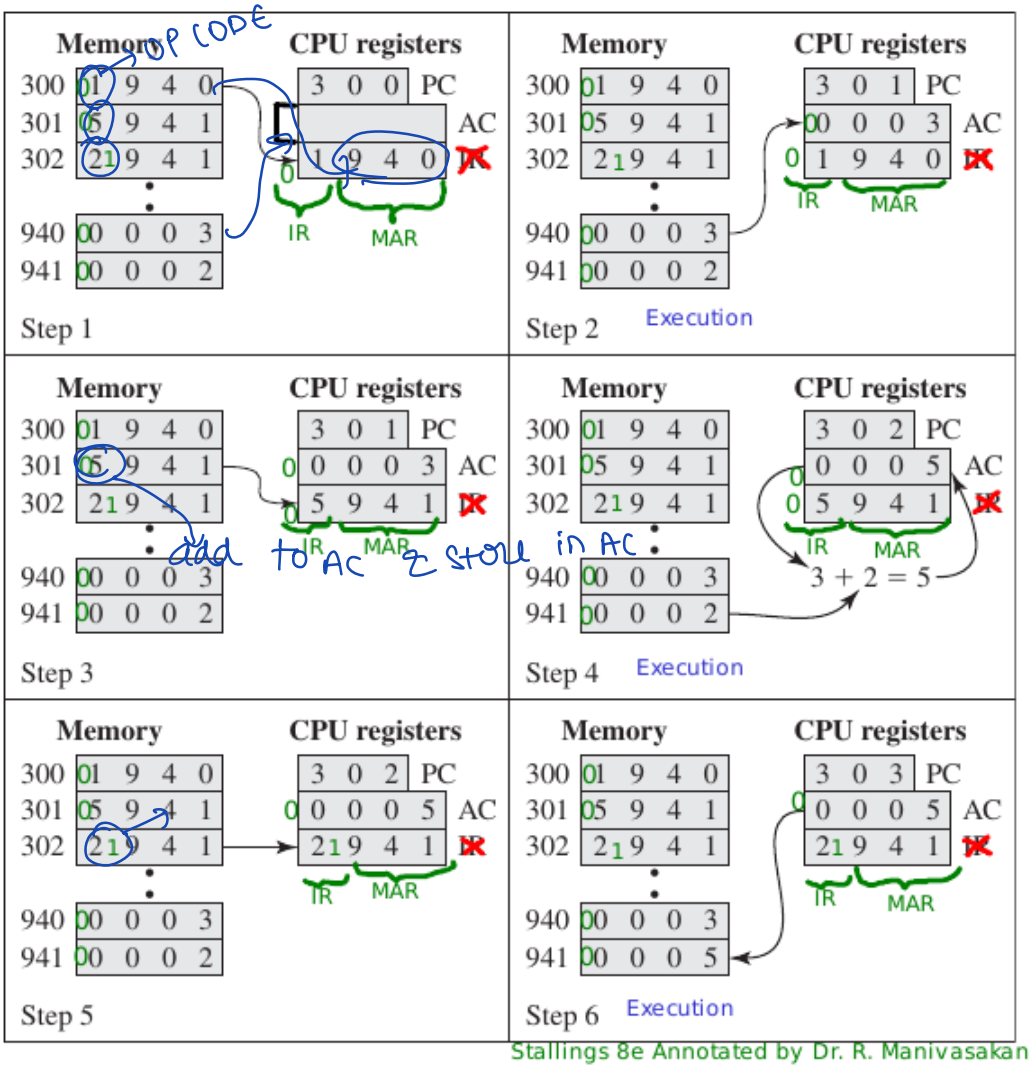


Fig 1: A program showing its execution in van Neumann's computer.

Following table gives the ISA for van Neumann first stored program computer.

Instruction Type	Opcode	Symbolic Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD −M(X)	Transfer −M(X) to the accumulator
	00000011	LOAD |M(X)|	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD −|M(X)|	Transfer −|M(X)| to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP+ M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD |M(X)|	Add |M(X)| to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB |M(X)|	Subtract |M(X)| from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2; i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

(a) List the instructions used in program shown in Fig 1 above. 940, 940, 941, 941, 941, 941

(b) Quantitatively evaluate the value of MAR in each clock cycle (steps 1 to 6 as shown above).

(c) What is the role of AC here? hold operands temporarily

3. The stray capacitance, frequency and supply voltage, in an intel microprocessor chip and an AVR microcontroller chip, respectively are given as below:

   (a) intel processor chip 10 nF, 1.72 V and frequency 6 GHz power.

   (b) AVR microcontroller chip 5 V, 100 pF operating at 1 MHz (0.25 W).

   Given the above, answer the following:

   intel: $P = CV^2f = 10^{-8} \times 1.72^2 \times 6 \times 10^9 = 177.5\,W$ AVR: $10^{-10} \times 5^2 \times 10^6 = 2.5\,mW$

   (a) Compute the heat (dynamic power) generated (which is to be removed or dissipated by cooling mechanism)

   Intel → 177.5 J/S AVR → 2.5 mJ/S

   (b) What is the heat removing rate (in J/sec) to be implemented by the cooling mechanism for each of the above, so that temperature of the chip doesnt increase?

   (c) While, any heat generated, less than a watt could be ignored (meaning, the heat generated, would either be naturally be dissipated or would not raise the temperature of the chip substantially), which of the above (intel processor or AVR microcontroller) doesn't need any cooling mechanism. AVR

4. **van Neumann versus Harvard Architecture - Bus Size:** Consider a 64-bit intel processor having a 32-bit address bus, with 4 control lines (corresponding to RD/WR, chip enable etc) is proposed to be connected to the main memory (housing program and data) through (a) van Neumann architecture (b) Harvard architecture. Compute the total number of wire-traces (system bus width) to be used in each of the above cases. Give the corresponding picture. [Note that this problem concerns only the part of the system bus which connects to memory, not the part connecting to I/O].

(a) van Neumann: 32+64+4 = 100 wires

(b) Harvard: instruction memory + data memory = 200 wires

(32+64+4=100) (32+64+4=100)

5. **van Neumann versus Harvard Architecture - Computational Time:** To see the advantage of Harvard architecture, consider an assembly program of an engineering task which is implemented in AVR processor (operating at 200 MHz) and intel processor (operating at 2 GHz). The RISC program in AVR contains 100 lines (each has external SRAM access of 1 clock cycle for operand, execution & writeout - concurrently ROM fetch of next instruction and so on). The CISC program in intel takes 50 lines, in which given an instruction following number of clock cycles are consumed: first 10 cycles are for instruction fetch, next 10 cycles are data (operand) fetch and next 20 cycles are for execution. The last 10 cycles are writeout (WO). Next instruction can be fetched during execution of previous instruction (primitive pipelining). Compute the time taken to execute for the whole program by the AVR and intel processor.

AVR $t = \frac{1}{2\times10^8} = 5\times10^{-9}$ s

100 lines

$t = 5\times10^{-7}$ s

Intel $t = \frac{1}{2\times10^9} = 5\times10^{-10}$ s

1st instruction → smooth: $\overbrace{20+20+10}^{50}$

next $\overbrace{20+20+10}^{30}$

$20 + 20 + 10$

$50 + 49\times30 = 1520$

$1520\times10^{-10}\times5 = 7.6\times10^{-7}$

EE2016 Microprocessors Theory and Lab, Aug - Nov 2025.

Tutorial 2 (Classes on 12, 13, 19 & 20th of August 2025)

Dr. R. Manivasakan, EE Dept, IIT Madras.

1 Fill in the blanks

1. The most common processor instruction (which occurs 90 % of the time) is ..data flow..... (data flow / addition / Ex-OR / multiplication).
2. The output of two or more gates should not be connected together, because ..it causes short circuiting.
3. Tri-state gates are solutions to ..preventing a short circuit when 2 different outputs from a logic gate are connected
4. A single bit data bus interfacing the N (N>3) flip-flops is created by ..... while the control signals involved are ..... To transmit data from one FF to another, the procedure is .....
5. The purpose of buffer registers connected to data bus is to ..temporarily hold / buffer data being transferred, often to sync data between devices of diff speeds
6. In N (=16) 8-bit registers connected through 8-bit data bus, the width of address bus is ..4....., width of data bus is ..8.....
7. Security of the processor is more in ..Harvard.. (Harvard / von Neumann architecture) and the reason(s) is (are) ..it has seperate memories and busses for program and data, which can prevent data injection attacks from corrupting program instructions
8. AVR microcontroller complies to ..Harvard.. (Harvard / van Neumann architecture). Hence, the program memory and data are stored in the ..different.. (same / different) memory location.
9. The purpose of the functions lines in ALU is to ..select the specific arithmetic / logical operation the ALU should perform.. Function lines are generated by ..control unit..
10. Exactly 1 KB means ..1024... bytes.
11. The binary and decimal equivalent of hex numbers 0x 2B90 is ..0010101110010000.. and ..$2\times16^3+11\times16^2+9\times16=11152$..
12. In Load / Store architecture (LSA) apart from LD and ST instructions other instructions ..can't..... (can also / can't) access the memory.
13. In LSA, ..either one.. (either one or both or none of) the operands ..can..... (can / can't) be from main memory.
14. In register-memory architecture (RMA), ..one/both.. (either one or both or none of) the operands ..can..... (can / can't) be from main memory.
15. Which of the following is (are) volatile? (a) SRAM, (b) EEPROM (c) DRAM (d) NV-RAM (Refer mazidi). V (used in CPU caches) NV (storing firmware like a computer's BIOS) V (most common type of memory used for the main RAM) NV (retains info when power is turned off, by using a small battery for backup power)
16. Of the instructions (a) ADD R0, R1 (b) MOV R15, R16 (c) LDI R0, K ..b, c.. is (are) data transfer operation(s), ..a.. is (are) ALU arithmetic operation(s).

volatile → memory requires constant power to maintain the stored information

2 Answer the following

1. Consider the AVR ATmega8 processor, which has the flash memory (program ROM) of 8KB. Given that, in AVR the word size in program ROM is two bytes,
   (a) what is the role of the PC in muP? holds memory address of next instruction to be fetched and executed
   (b) is the PC (a register), accessible to programmer? If it is so, what is it's address in ATmega8? Where does it sit? Within the muP or external SRAM or ROM? (No) (within µP core)
   (c) what is the memory of flash in terms of bytes? What is the maximum number of instructions, that can be accommodated in the given program ROM? (8x1024=8192 bytes) 8192/2 = 4096
   (d) What is the size of PC (in bits)? 12 bits ($2^{12}=4096$)
   (e) What is the size of program address bus? = 12 bits (same as PC size)

2. A 512KB memory chip has 8 pins for data. Find
   (a) the organization (meaning, how many words versus word size) and 512x1024 bytes → 8 bits = 1 byte ⇒ 512x1024 words 512x1024 lines
   (b) the number of address pins for this memory chip $2^9 \times 2^{10} \Rightarrow 19$ pins

3. In a given byte-addressable microcomputer, memory locations 0x 10000 to 0x 9FFFF are available for user programs. The first location (address) is 0x 10000 and the last location (address) is 0x 9FFFF. Calculate,
   (a) the total amount of memory (in bytes) available for user programs 589824 (0x9FFFF − 0x10000 = 0x8FFFF + 1 = 0x900000; 9x16^4)
   (b) the word length 1 byte = 8 bits
   (c) the size of address bus $\log_2(589824) \approx 20$
   (d) the size of data bus 8 bits

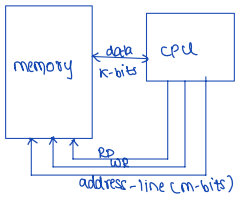
4. Consider a 1-bit memory element constructed out of a flip-flop. Assume active high logic. Can the control signal "WR / latch enable" and "RD enable", both high at the same time? Both of them low at the same time? (Yes, they can be low at the same time (allows memory element to hold its data without interfering with other components of the same bus. No, they can't be high at the same time (memory's preparing to store new data from the i/p line and placing its currently stored value onto the output line))

5. How is a single 8-bit register built out of SR flip-flips? Show the control lines to the register and show how is it connected to RD & WR of the individual flip-flops. Mark the data lines.

6. Given a set of n $(= 2^m)$ registers (each of (word) length k bits) for some $m = 0, 1, 2, \ldots,$
   (a) What is the memory size in bits? And in bytes when $k = 8$? $n \cdot k$
   (b) What is the minimum number of address bits necessary to pick an unique register from the above set? m
   (c) Evolve an addressing scheme which uses minimum bus size. Show the digital circuit derived indicating the TT if any, at all used in the design. $m \to 2^m$ decoder

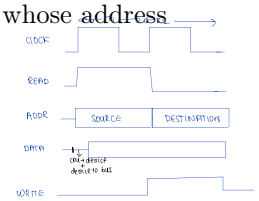
A_1	A_0	E_0	E_1	E_2	E_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

   (d) Give an overall block diagram indicating the various units used in (b) above viz., register bank, address bus, data bus and control lines / bus. Indicate the size of each bus.
   (e) Instead of separate "WR / latch enable" and "RD enable" physical wireline, what if one has a single physical line along with the 'rule', READ for logical 'low' and WR for logical 'high'. What are the pros and cons of such a scheme? one line in ctrl bus is reduced, resulting in reduced footprint. but both of them can't be 0 at the same time, hence no idle state
   (f) Explain how the steps involved in instruction MOV Rd, Rs when executed, by using the above block diagram. Here, Rd & Rs are registers of size l bits. Identify the physical path of the digital signal journey. mov → 2 clock cycles
   (g) Show the timing diagram of execution of the instruction in (d). [Timing diagram has not been taught in the class and hence this is optional].

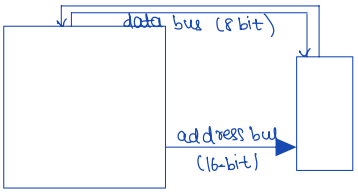


(a)
Read from source:
1. Addr path: CU places addr of Rs onto the m-bit address bus
2. Ctrl Path: CU asserts the Read signal
3. Data path: Rs is selected by the decoder and places its k-bit content onto the data bus
(b) write to destination:
1. Addr path: CU places addr of Rd onto m-bit addr bus
2. Ctrl path: ctrl unit asserts the write signal
3. Data path: CPU places the data it was holding in data bus; Rd then latches this data completing the move

7. Consider a 8-bit processor in which the data bus is connected to internal 8-bit registers whose address range from 0x 0000 till 0x FFFF.



CPU's C.U. places address of R1 onto 16-bit address bus. C.U asserts the read control signal. The register bank's internal decoder activates register R_1, which places it 8-bit data onto the data bus. The CPU reads this data from the bus into an internal, temporary holding register. The CPU's C.U. then places the address of R0 onto 16-bit address bus. CPU places the data it was holding onto the 8-bit data bus. The C.U asserts the write control signal. The decoder selects register R0, which latches the data from the data bus.

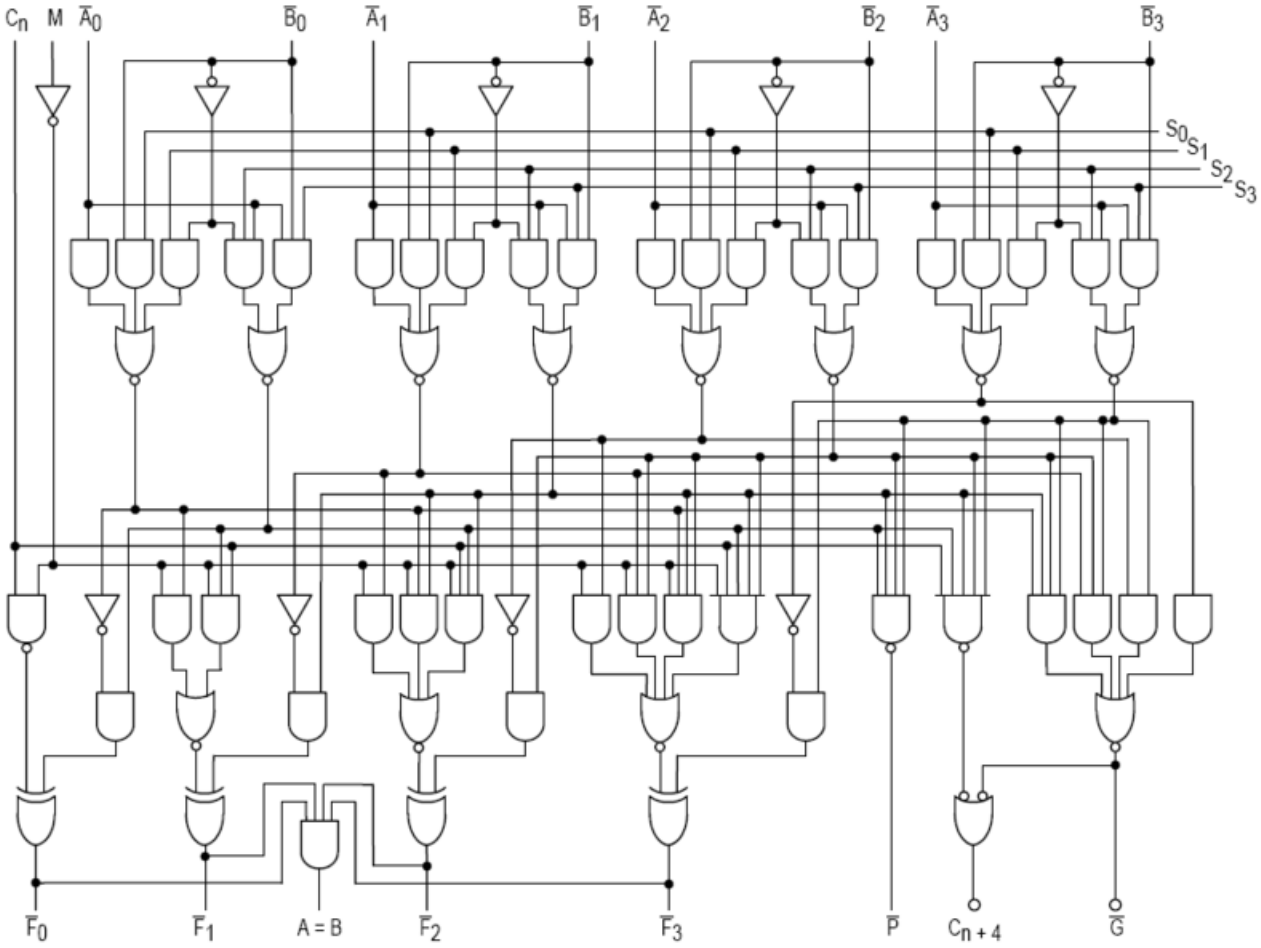


(a) Show the block diagram as well as circuit diagram of the above system, showing the registers, data bus, control lines and address bus. Indicate quantitatively the various system parameters.

(b) What is the word length here? What is the size of address bus and data bus?

(c) Show the hardware level steps to execute the instruction `MOV R0, R1`, given that the registers R0 & R1 are at addresses `0x 0000 & 0x 0001` respectively. Show the control signals, timing diagram in the digital schematic of memory, address bus, data bus and control lines.

8. Shown below is the circuit diagram of 1960's famed 4-bit ALU 74181 MSI.



Consider the following table of inputs:

					Active-high logic	Active-high logic
	S_3	S_2	S_1	S_0	Logic $(M = 1)$	Arithmetic $(M = 0)(C_n = 1)$
	0	0	0	1	output F = ?	output F = ?
	0	1	1	0	output F = ?	output F = ?
	1	0	0	1	output F = ?	output F = ?
	1	1	0	0	output F = ?	output F = ?

Assuming active-high logic, enter the value of $F_3F_2F_1F_0$ in terms of $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ for the cases given in the above table. You could use the short hand notation of A for $A_3A_2A_1A_0$. Give the pathway of the digital signal as it traverses from the input to output.

add^n: let $q_1 \geq q_2$
$\Rightarrow N_2 = c_2 \times 10^{(q_2 - q_1)} \times 10^{q_1}$
add the coeffs: $(c_1 + c_2 \times 10^{q_2 - q_1}) \times 10^{q_1}$
Normalize

sub^n → same as add^n

mult: $c_1 \times c_2$
$q_1 + q_2$
$(c_1 \times c_2) \times 10^{q_1 + q_2}$
normalize

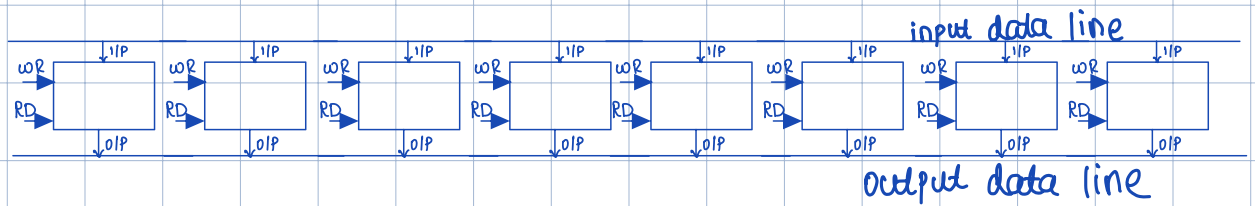
div: c_1 / c_2
$q_1 - q_2$
$(c_1/c_2) \times 10^{q_1 - q_2}$
normalize

normalize: $1 \leq c < 10$

9. Floating Point Unit (FPU):

(a) Given the two positive floating point numbers (in IEEE 754 format), $N_1 = c_1 \times 10^{q_1}$ and $N_2 = c_2 \times 10^{q_2}$, give the table containing addition, subtraction, multiplication and division operations on these numbers.

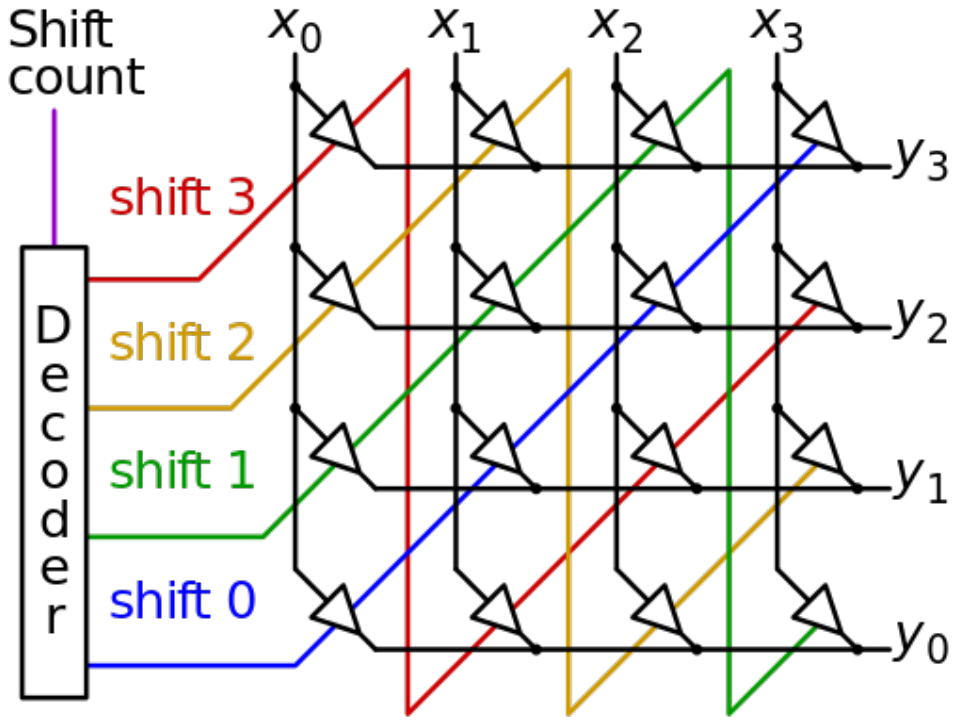
5. How is a single 8-bit register built out of SR flip-flips? Show the control lines to the register and show how is it connected to RD & WR of the individual flip-flops. Mark the data lines.



(b) Build an FPU unit, specify the internal circuitry of adder, subtractor, multiplier and specify the and internal interconnection using various internal bus.

14. [Not included in portions for Quiz1] Barrel shifter:

The anatomy of a 4-bit crossbar barrel shifter is given below:



Consider the clock interval is T_{clk} secs.

(a) The above crossbar barrel shifter circuit is supposedly to be a combinational circuit. Redraw the above diagram with ALL elements being explicitly expressed as digital logic gates (**Hint:** Strictly speaking the diodes here, doesnt make a logical gate. It is just a symbolic representation that the output of decoder controls the diode, whether to pass the digital data from input (column) to output (row). [Diode representation is from the crossbar terminology used in telephony]. Replace the diodes by a suitable logic circuit with the output of the decoder plus the input (X_i). Take care of the problem of connecting together the output of more than one digital gate or FF or combination thereof).

(b) Given the 4-bit shifter implemented through a sequence of FFs, compare the performance of barrel shifter for time it takes for 3-bit shift, with the time taken by 4-bit FF shifter for the same job.

(c) Mention the type of shifter circuit is the above barrel shifter? Shift right or left or rotate left or rotate right? Can you modify the one above (whatever type) to the other type? If you think it is possible, redraw the modified circuit. If you think it is NOT possible, then give reasons, why it is not possible.

(d) Can we modify the above circuit to implement the following

i. shift right
ii. shift left
iii. arithmetic shift right
iv. arithmetic shift left
v. left logical shift
vi. right logical shift
vii. left rotate through carry (carry is assumed available)
viii. right rotate through carry (carry is assumed available)

EE2016 Microprocessors Theory and Lab, Aug - Nov 2025.

Tutorial 3 (Classes on 13, 15, 18, 19 & 20th of Aug, 2025)

Dr. R. Manivasakan, EE Dept, IIT Madras.

Portions: Wk5-Start ALU, shift register, flag register and their interconnection through address, control and data lines, ALU - components (logical & arithmetic operation engines), FPU, addressing modes (immediate, direct, indirect, register direct, register indirect), illustration of MOV in hardware, LDI ditto, assembly language CU, EUCI, PFCI, EUCL, PFCL, stack pointer, link register, EU & CU put together, AVR hardware architecture (Mazidi), RISC-style & CISC-style assembly programming.

1 Fill in the blanks

1. Instruction size is ....fixed.. (fixed / variable) in AVR. → RISC
2. The number of general purpose registers (GPRs) within the AVR processor is .....32.....
3. The D7 bit of SREG in AVR denotes ....Global Interrupt Enable (I) flag
4. In mask ROM, the contents are programmed by the ..manufacturer.. (manufacturer / user).
5. In programmable ROM, the contents are programmed by the ..user..... (manufacturer / user).
6. In EPROM, selectively a byte ..can't.. (can / cant) be erased and rewritten.
7. In Electronically Erasable PROM (EEPROM), selectively a byte ..can.... (can / cant) be erased and rewritten.
8. Flipflops are used to construct ..SRAM..... (SRAM / DRAM).
9. Capacitors are used to construct ..DRAM..... (SRAM / DRAM).
10. Volatile SRAM is made up of ....FFs.. (FFs / semiconductor capacitors) and is made to be non-volatile by using ..batteries
11. The advantage(s) of DRAM over SRAM is (are) ..less power consumed, cheaper
12. The advantage(s) of SRAM over DRAM is (are) ..faster, no refreshing cycles req.
13. The pair R27 R26 hold ..16 bit address in the main memory and could be used to uniquely pull one location out of 2^16 locations.
14. Stack pointer (SPH & SPL) is located in ..I/O registers area.. (GPR region or I/O registers area).
15. Enumerate the difference between IN and LDS instructions.
16. Instructions that affect all the six flag bits C, H, Z, S, V and N are ...ADD, SUB.. (ADD, AND, DEC, SUB, INC). (S: sign, Z: zero, V: overflow, N: negative, H: Half carry, C: carry; DEC, INC, AND don't affect C & HC; 1° arithmetic instructions (ADD, ADC, SUB, SBC))
17. The difference between arithmetic shift left and logical shift left is (are) ...none.....(Right shifts matter) (LSR: unsigned /2, ASR: signed /2)
18. Every assembly instruction consist of two parts: first part is ..OPCODE..... and the second one being ..operand field.. (OPCODE: and that tells CPU what action to perform; operand field: what data to act upon)

19. The EUCI .....doesn't..... (does / doesn't) change the sequence of program execution.
execution unit control instructions
20. The PFCI .....does..... (does / doesn't) change the sequence of program execution.
program flow control instructions
21. The hardwired control unit consists of ....., ... and ..... → control circuit matrix
finite cyclic pulse train generator
instruction decoder

2 Answer the following

1. The address bus of a computer has 16 address lines, $A_{15} - A_0$. The hexadecimal address assigned for a given device, is 7CA4. 0111 1100 1010 0100

16-bit AND gate, all zeros inverted before connection

   (a) Draw the address decoder (digital circuit). [Is it combinational circuit only or it is a combination of sequential & combinational logic?]

   (b) At a later point in time if the above address decoder is modified such that the device ignores lines A_8 and A_9, what are all the addresses to which this device will respond? [Mention the range of addresses]. 0x9CA4, 0x98A4, 0x7EA4, 0x7EA4

2. Consider the problem of usage of stack and stack pointer in the form implemented in AVR ATmega8. Recall the picture on stack and stack pointer. Information:

   (a) The following is the list of data PUSHed onto stack during an interrupt (a) PC value (2 bytes) (b) SREG (1 byte) (c) GPRs (3 bytes) (d) stack pointer itself (2 bytes) (e) context data 1 (function arguments only for subroutines, 2 bytes) (f) local variables (2 bytes). Thus, the stack frame for a subroutine call (SC) is 12 bytes in length and for an interrupt, it is 10 bytes, which is PUSHed upon entry and POPed out upon from their 'exit'.

   (b) Stack frame is memory allocated in bytes, on the stack for a single subroutine call or interrupt call.

   (c) Stack Overflow: A critical error that occurs when a function call (or interrupt) attempts to PUSH data into the stack, but there is no more memory available.

   (d) Stack Capacity: The total available memory for the stack. Assume the stack in AVR 128 bytes

① initialize:
cur-depth = 0
max-depth = 0
stack-cpcty = 128
framesizeS = 12
framesizeI = 10
overflow occured = False

② Process: read sequence from left to right, one event at a time. For each event, update current depth based on:
(i) event is CS: cur-depth += framesizeS
(ii) " " CI: " " framesizeI
(iii) " " return: " -= framesizeS
(iv) " " RI: " " framesizeI

③ check for overflow & update max-depth
if cur-depth > stk cpct = overflow = True
if cur-depth > max-depth = max-depth = cur-depth

④ return max-depth
overflow occurred

The following sequence(s) of interrupts or subroutine combinations are presented to the AVR microcontroller. Legend: CS - Subroutine call, CI0 - AVR interrupt INT_0, CI1 - AVR INT_1, CI2 - AVR INT_2, R - return (from both interrupt and subroutine), RI - RETI in AVR

   (a) CS CS R CS R R

   (b) CI2 CS CI1 CS CS R R RI CI0 RI R CS R CS R RI

   (c) CI2 CS CS CS CI1 CS CS CS CI0 CS CS CS R R R

Problem Statement: For each sequence above,

   (a) Write an algorithm to determine the maximum stack depth reached during the execution of each of the above three given sequences. You also need to detect if a stack overflow occurs at any point.

   (b) Provide

      i. maximum stack depth

      ii. final statement on whether a stack overflow has occurred.

Hint: Use the following variables - Variable definitions:

- `current_depth`: an integer to track the deviation of stack pointer from the top location (position)
- `max_depth`: an integer to keep track of maximum deviation of stack pointer from the top location
- `stack_capacity`: 128 bytes
- `frame_sizeS`: 12 → subroutine
- `frame_sizeI`: 10 → interrupt

$t = 1/250\times10^6 = 4\times10^{-9}$ s Program time $= 2\times10^6 \times 20 \times 4\times10^{-9} = 16\times10^{-2}$ s

Polling: No of polls $= \frac{160\times10^{-3}\text{ s}}{0.125\times10^{-3}\text{ s/poll}} = 1280$ polls $\Rightarrow$ polling overhead $= 1280 \times 2\times10^{-6}$ sec/poll $= 256\times10^{-5}$ s $\rightarrow t = 16\times10^{-2} + 256\times10^{-5} = 1.625600\times10^{-2}$ sec

Interrupt: overhead per interrupt $= 8\text{ns} + 4\text{ns} = 12\times10^{-9}$ s $\Rightarrow$ interrupt overhead $= 2\times12\times10^{-9}$ s $\Rightarrow t = 1.600000024\times10^{-2}$ s

↓ more efficient

3. Polling and interrupt: Consider the problem of interfacing a keyboard with an intel microprocessor operating at 1 GHz, given that a maximum of 1 key press per 20 msec (recall keyboard debouncing problem). The above interfacing could be done in two ways: (1) using polling KIN bit in KBD_STATUS and (2) using interrupt through KIRQ bit (in KBD_CONT) implementation. To see the performance difference between the polling and interrupt, assume there is only one key press within an interval of 80 ms. Given that the CPU is running an user program of 1 million instructions and each instruction (CISC) is taking 20 clock cycles, what is the total time for completion of whole program in both cases (of polling & interrupt). Ignore the time of reading from the KBD_DATA register into the CPU and the interrupt latency of 4 ns.

250 MHz 2 key presses 160ms 4ns 4ns polling cycle → 0.125msec, polling pulse width = 2μs

4. **Priority Among Interrupts:** A processor has seven interrupt-request lines, INTR1 to INTR7. Line INTR7 has the highest priority and INTR1 the lowest priority. Design a priority encoder circuit (show truth table, Karnaugh's map & expression for output(s)) that generates a 3-bit code representing the request with the highest priority. See https://en.wikipedia.org/wiki/Priority_encoder.

5. **RISC-style versus CISC-style Assembly Program (Optional):** Problem is the addition of two numbers in two distinct memory locations in the main memory and storing the result in a third memory location. SRAM data memory locations in a RISC (AVR) processor is 0x00A0 and 0x00A1. Final storing location is 0x00A2. Store the carry (if at all, is generated), in 0x00A3. Write a RISC style assembly program (use AVR ISA). Similarly identify a CISC processor like intel and write a CISC style assembly program for the above task. Compare the above two in terms of the following

   (a) How many clock cycles each of the above processors take?

   (b) How much is the memory occupied by their respective assembly programs?

   (c) Given the average frequency of an AVR processor being 16 MHz and that of intel processor being 3 GHz, compute the total time for the above program, in each of the above cases.

$t = \frac{1}{250}\times10^{-6}$ s

2 key press per 16×10^{-2} sec

2×10^6 instructions → 20 clock cycles each

$\frac{1}{250\times10^6} \times 2\times10^6 \times 20 = 160\times10^{-3}$ sec

polling overhead: $\frac{160}{0.125} \times 2\times10^{-6} = 2560\times10^{-6}$ sec

1625.60×10^{-6} → polling

$160\text{ms} + (4\text{ns} - 16\text{ns}) \times 2$ → interrupt

EE2016: Microprocessor theory and lab

Tutorial-1 (TGV)

Q1. Which addressing mode uses operands that are part of the instruction itself?

A) Direct

B) Register

C) Immediate

D) Indirect

operant is a constant value embedded directly into instruction

Q2. In `MOV AX, [BX]`, the addressing mode for the source operand is:

A) Register Indirect

B) Direct

C) Immediate

D) Indexed

[BX] indicates the address pointed by the BX register.

Q3. The addressing mode where the operand address is explicitly given in the instruction is:

A) Immediate

B) Direct

C) Register Indirect

D) Implied

Q4. For the instruction `PUSH AX`, the addressing mode used for the operand `AX` is:

A) Implied

B) Register

C) Direct

D) Immediate

Q5. In indexed addressing mode, the effective address is calculated as:

A) Base Register + Index Register

B) Index Register + Displacement

C) Program Counter + Displacement

D) Stack Pointer + Offset

Q6. Relative addressing mode is primarily used for:

A) Data transfer

B) Branch instructions

C) Arithmetic operations

D) I/O operations

Q7. The addressing mode where the operand is implied by the instruction (e.g., **STC**) is:

A) Immediate
B) Direct
C) Implied
D) Register

The STC instruction sets the carry flag.

Q8. For MOV AX, [BP+10], the source operand uses:

A) Base-Indexed
B) Based Relative
C) Indexed
D) Register Indirect
BP is the base pointer register.

Q9. Consider an 8086 system where:

DS = 4000H

BX = 0200H

DI = 0040H

0250H

Memory location 40240H contains 1A2BH.

The instruction MOV AX, [BX+DI+10H] is executed. What value is loaded into AX?

A) 0200H
B) 0040H
C) 1A2BH
D) 40240H

Q10. In a processor with Base-Register Addressing, if the base register contains 3000H and the instruction

specifies a displacement of +200H, what is the effective address? What happens if the base register is

incremented to 3002H before instruction execution?

A) 3200H, 3202H

B) 3200H, 3200H

C) 3000H, 3002H

D) 3200H, 3202H with auto-indexing

Q11. Consider a 32-bit processor where:

EAX = 00001000H

EBX = 00002000H

Memory location 00003000H contains 00004000H

After MOV ECX, [EAX+EBX*2] , what is ECX?
A) 00003000H
B) 00005000H
C) Content of 00005000H
D) Content of 00003000H

Q12. In a RISC-V processor, the instruction 1d t0, 8(s1) uses which addressing mode? What is the role of s1 and 8?

A) Register Indirect; Base = s1, Offset = 8

B) PC-Relative; PC = s1, Offset = 8

C) Indexed; Index = s1, Base = 8

D) Immediate; Value = 8

Refer to the RISC-V instruction set using the internet to answer this question.

Q13. In an 8085 system, LDAX B uses:

A) Register Indirect
B) Direct
C) Immediate
D) Implied

Refer to Intel 8085 instruction set using the internet to answer this question.

Q14. Which addressing mode cannot be used for destination operands in typical ISAs?

A) Immediate
B) Register
C) Direct
D) PC-Relative

Q15. In an 8086 system, the following instructions are executed:

Assembly:

MOV AX, 1234H

MOV [2000H], AX

LES DI, [2000H]

If DS = 0100H, what are the values of ES and DI after execution?
A) ES = 1234H, DI = 0100H
B) ES = 0100H, DI = 1234H
C) ES = 1234H, DI = 5678H
D) ES = 5678H, DI = 1234H

Q16. Consider a microprocessor with a 20-bit address port. Suppose the opcode field takes 8 bits, then

What will be the length of an instruction if it follows (a) 4-address instruction format, (b) 2-address instruction format.

Anandhamayae Chythinyamayae Sathyamayae Premae

EE2016 Microprocessors Theory and Lab, Aug - Nov 2025. TB Edited

Tutorial 4 (Classes on 1st, 3rd, 8th Sep - 1st batch (29th Aug, 2nd, 9th Sep))

Dr. R. Manivasakan, EE Dept, IIT Madras.

Portions: Keyboard debounsing problem, polling implementation in KB debouncing problem, interrupt implementation in KB debouncing problem, and polling (KIN bit) and interrupt (KIRQ) implementation in muP - KB PIC problem, CU design, microprogramming and hardwired control unit, hardwired CU design examples: #1 Booths multiplier, #2 GCD processor, hardware design procedure - abstraction of CU as an FSM - Moore's (rather than Mealy's) machine, consolidation of states involved, table of current state versus next states and value of output signals (Boolean variables), translation of above states into excitation table (TT) and subsequent digital design.

1 Fill in the blanks

1. The two main types of addressing a peripheral (given a set of peripherals, picking a desired one) are memory mapped IO..... and ..port mapped IO.....
2. In memory mapped I/O the address space is ..shared... (shared by / separate for) memory and PIC registers of peripherals.
3. Example of port mapped I/O based processors is ..Intel 8085 microprocessor.
4. Three important registers in PIC of a controller are ..data register.., ..status register and ..control register
5. In a byte addressible memory chip of size 256 bytes, the word length is ..1 byte.. the address bus width is 8 bits data bus width is 8 bits and the control pins are ..CS (CHIP SELECT)., ..RD., ..WR and ..CLK..
6. The status signal (in the context of PIC) is unidirectional (unidirectional / bidirectional) and if it is unidirectional, then its direction is from ..PIC.. to ...CPU..
7. For a keyboard the system data bus connecting to CPU is unidirectional (unidirectional / bidirectional) and if it is unidirectional, then its direction is from ..keyboard to ...CPU.....
8. In polling method of handshake, the proactive component is ..CPU..... (CPU / Peripheral)
9. Advantage of interrupt over polling is improved CPU efficiency
10. Prioritizing the service by microprocessor, for such service requests from various I/O devices, in polling, is ..impossible. (possible/ impossible), while in interrupts it is .....possible..... (possible/ impossible).
11. The criteria for concluding that "a key has been pressed", (in the polling method of solution to the debouncing problem) is ...13..... number of samples of '0' (no contact) followed by ....7..... number of samples of '1' (contact).
12. PC value abruptly jumps during instructions: JMP, subroutine interrupt (select from following list: LDI, JMP, subroutine, ADD, interrupt).
13. The role of stack pointer in interrupt is (are) ..save context of program (PC, registers) onto stack before executing ISR

14. Stack is a ..LIFO..... (FIFO or LIFO or FILO or LILO) system.

15. The number of POP and PUSH operations in a stack, for a specific interrupt are ..equal (equal / necessarily unequal).

16. The stack pointer, points to variable bottom most location (variable bottom most location / variable midlocation / fixed top most location) of active area of stack, which uses the convention of PUSHing from top.

17. The purpose of KIN bit in the KBD_STATUS register of keyboard PIC is ..of KBD data is fresh.. (to tell CPU if data in data register is keyboard input)

18. The KBD_STATUS register can only be written into (read / written into) by keyboard peripheral.

19. The KIN bit in KBD_STATUS register is used for .both..... (polling / interrupt / both polling & interrupt schemes) method of handshake.

20. Order the rates in ascending order: (a) rate of keys pressed (b) rate of CPU cycles (c) rate at which the display device could display characters ...ascsb.....

21. The meaning of the KIN bit in KBD_STATUS being '1' is that the data (ASCII code) in KBD_DATA is ..valid..... (valid / invalid - valid means the CPU would READ, invalid means CPU has already READ and should NOT READ again).

22. The main difference between a subroutine (synchronous (planned, initiated by the program)) and an interrupt-service routine (asynchronous (unplanned, triggered by external event)) is ..... (Hint: in terms of their arrival time?) during execution of a program.

23. The hardwired control unit consists of ...mapper... (instruction) and ..combinational and sequencial circuit.....

24. Booths algorithm is to compute multiplication (addition, multiplication, Ex-OR) of two numbers.

25. Final derived circuit for CU for Booths algorithm consists of ..... (purely combinational, purely sequential, a combination of combinational and sequential) digital circuit.

2 Answer the following

1. Hardwired control is used in contemporary instruction-set microprocessors. Consider the design of a hardwired control unit based on the classical method for a gcd processor that computes the gcd(X, Y) of two positive integers X and Y . The gcd procedure assigns the input data X and Y to a pair of registers XR and YR. While XR $>$ 0, the procedure (i) swaps XR and Y if XR $\leq$ YR and (ii) subtracts YR from XR and assigns the result to XR. The GCD procedure ends whenever XR<=0, yielding the GCD (as the final value of YR).

   (a) Show the hardware to implement the GCD algorithm.

   (b) Show the datapath hardware needed to implement the above GCD procedure.

   (c) Show the CU with input and output control signals for the above datapath hardware. (→ Select, Swap, Subtract, loadX, loadY, Done; (XR>0), (XR≥YR))

   (d) Derive the CU for the above GCD processor by the standard procedure (taught in the class, steps given below) after assuming Moore type FSM. (S_0 → Begin, S_1 → Swap, S_2 → Subtract, S_3 → End)

      i. Identify the right FSM modelling the algorithm (or pseudocode)
      ii. Identify the states and write the corresponding state table
      iii. Identify the excitation table (Truth table)
      iv. (Optional) Using Karnaugh's map derive the expression for each output
      v. (Optional) Using the result in 1(d)(iv) draw the corresponding circuit

2. Hardwired CU for Booths multiplier:

   (a) Give the Booths multiplication algorithm [Refer Stallings / ppt file uploaded. A modified version of the algorithm is implimented in majority of the processors available in the market even today].

Initialization:
Accumulator set to 0
multiplicand loaded to register M
multiplier loaded to register Q

Iteration: Repeat the following until counter is 0
· examine bits Q_0 (last bit of Q) & Q_{-1}
· if bits are 01 → A = A+M

(b) Show the hardware to implement the Booths algorithm.

(c) Show the datapath hardware needed to implement the above Booths algorithm.

(d) Show the CU with input and output control signals for the above datapath hardware.

(e) Derive the CU for the above GCD processor by the standard procedure (taught in the class, steps given below) after assuming Moore type FSM.

i. Identify the right FSM modelling the algorithm (or pseudocode)

ii. Identify the states and write the corresponding state table

iii. Identify the excitation table (Truth table)

iv. (Optional) Using Karnaugh's map derive the expression for each output

v. (Optional) Using the result in 2(e)(iv) draw the corresponding circuit

3. (Optional) Repeat the Pr. 2 for Floating Point Unit. Meaning consider the two floating point numbers with fixed finite precision for matissa and exponent (& thus memory to hold those numbers are fixed). Now, repeat the Pr. 2 for multiplication of two floating point numbers.

4. **Polling versus Interrupt:** In this problem, we consider two designs of handshaking procedure between CPU and a peripheral device: polling and interrupt. The CPU was executing its own program, till 32nd instruction. 33rd instruction requires a job be assigned to a peripheral device [Good example of a peripheral is the printer] by the CPU. Method by which the peripheral informs the CPU that it is done, and what the CPU doing in the intervening period is the subject of this problem. Three schemes analysed in this problem are: (a) CPU waits perpetually, for the task to be completed by the peripheral (without *polling & interrupt*) (b) *polling:* CPU concurrently executes subsequent instructions, but polls periodically to see whether peripheral has completed its assigned job and (c) *interrupt* CPU concurrently executes subsequent instructions, but is interrupted by I/O (whenever it needs service or during service, whenever it is done). The figure below illustrates the comparison between the schemes as they all evolve in time

   Following are the three cases in which a processor - peripheral handshaking is designed.

   (a) *Case A (Without interrupt & polling):* In this case, after initiating the peripheral I/O operations in T_{REQ}, the processor waits for the peripheral's I/O operation to complete. During this waiting period, the microprocessor perpetually checks whether the above operation is complete or not. Each such checking takes a time interval of T_{chk}. If the I/O job, (which takes $T_{I/O}$ secs) is over, the time T_{RES} is taken by the peripheral's I/O module for writing the result of the peripheral operation and the CPU reading it to conclude the I/O operation.

   (b) *Case B (Polling):* In this case, the processor is allowed to do other jobs (after initiating the peripheral's I/O operations in T_{REQ}), but in this scheme, the processor polls, at regular intervals $T_{poll} = N_2 T_{INS} + T_{chk}$ to check whether the I/O job is over or not. The checking time interval is T_{chk} secs.

   (c) *Case C (Interrupt)* In this case also, the processor is allowed to do subsequent instructions (concurrently with the peripheral's operation, after initiating the memory I/O operations in T_{REQ}). The difference is, the CPU can continue to execute the program, say, instruction 34, 35 and so on, till the peripheral '*interrupts*' it, to say it is done with the I/O task assigned. [If the I/O job taking $T_{I/O}$ secs is over, the processor would be interrupted by the concerned peripheral]. The time T_{RES} is taken by the I/O module for writing the result of the peripheral operation and the CPU reading it to conclude the I/O operation.

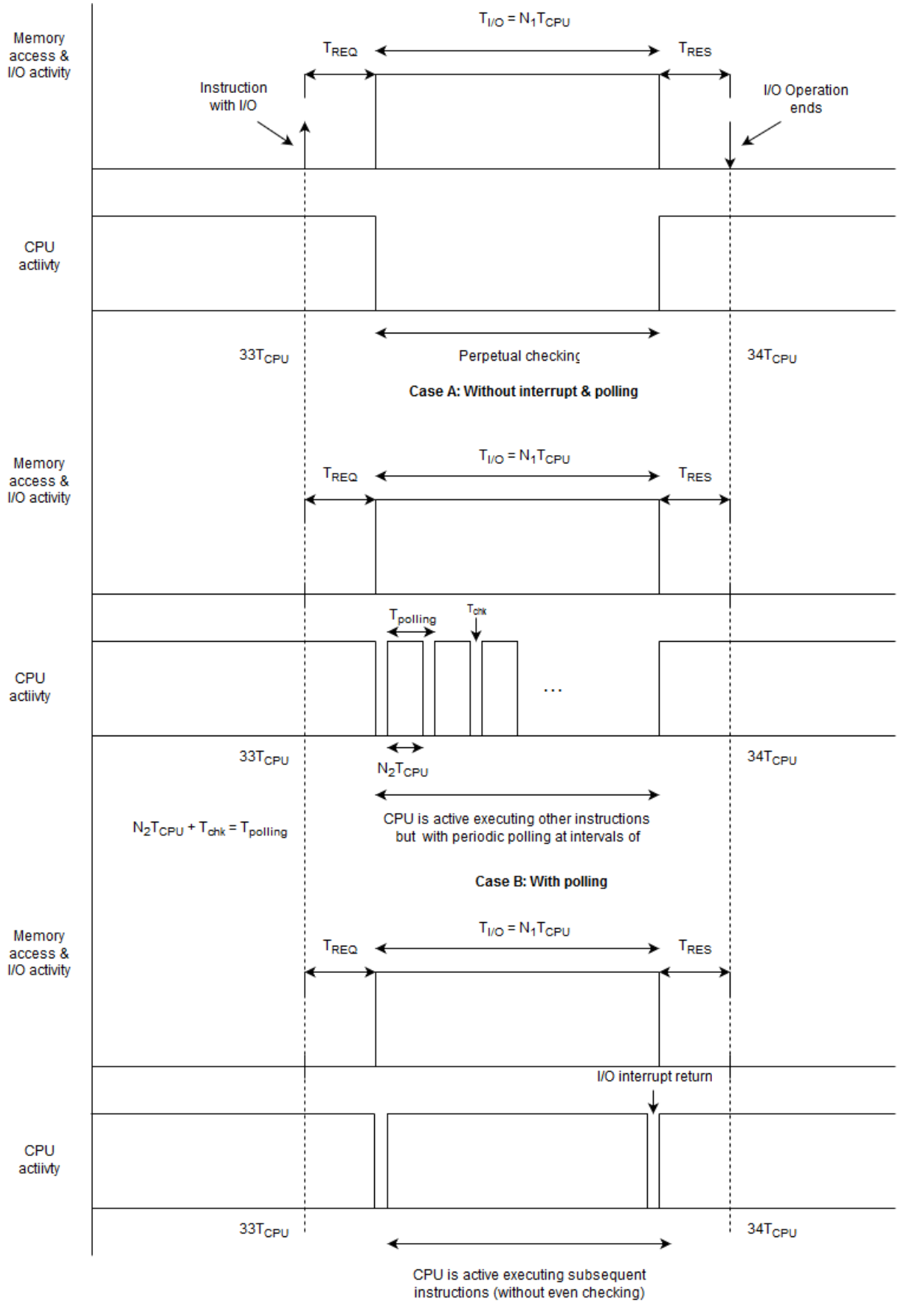
   Given the following assumptions,

   (a) For normal instructions (without I/O) the time required for decode and execute instructions is denoted by $T_{INS} = 6T_{clk}$.

   (b) There are a total of 100 such instructions (each of which takes T_{INS}) out of which the 33rd instruction (& only one such) involves I/O operation.

(c) Execution time for I/O operations is composed of three components (a) preparation for I/O operation which involves CPU time T_{REQ} (b) actual I/O (eg. print job) operation time being $T_{I/O} = N_1 T_{INS} \quad N_1 > 1$ (c) response by the I/O peripheral in setting the flag in PIC (whether the operation is success or failure) taking time T_{RES}.

(d) The time for preparation for I/O operation T_{REQ} represents time for exchange of protocol (hand shaking) control information initially between CPU and the peripheral. For the peripheral being output devices (like memory module, image printing in a color printer, video card etc), T_{REQ} means the time required to transfer data, for eg. time to transfer the digital image data to be printed through the peripheral, printer and the parameters of the print command. For the peripheral being input device (eg, camera, sensors, scanners, etc.,) T_{REQ} may mean initial control information from device, like the type of service requested, amount of memory space required, etc.


Memory access & I/O activity
$T_{I/O} = N_1T_{CPU}$
T_{REQ}
T_{RES}
Instruction with I/O
I/O Operation ends
CPU actiivty
$33T_{CPU}$
Perpetual checking
$34T_{CPU}$
Case A: Without interrupt & polling
Memory access & I/O activity
$T_{I/O} = N_1T_{CPU}$
T_{REQ}
T_{RES}
$T_{polling}$
T_{chk}
CPU actiivty
...
$33T_{CPU}$
N_2T_{CPU}
$34T_{CPU}$
$N_2T_{CPU} + T_{chk} = T_{polling}$
CPU is active executing other instructions but with periodic polling at intervals of
Case B: With polling
Memory access & I/O activity
$T_{I/O} = N_1T_{CPU}$
T_{REQ}
T_{RES}
I/O interrupt return
CPU actiivty
$33T_{CPU}$
$34T_{CPU}$
CPU is active executing subsequent instructions (without even checking)
Case C: With interrupt

(e) The I/O operation time $T_{I/O}$, may mean for instance, the physical printing time. In many electromechanical peripherals, time of execution is substantially larger than that of clock interval. In the case of, memory transfer among memory chips, the memory transfer happens between the data register of the PIC and the CPU main memory. For input devices like camera, sensors, scanners, etc., it is the actual data transfer of enduser data from PIC to CPU. For scanner, physical scanning and data being available in its PIC.

(f) The time interval T_{RES} accounts for again the protocol (hand shaking) control overhead for concluding the I/O transaction. For a color image printer $T_{REQ} \gg T_{RES}$. In this problem, for cases A through C, take $T_{REQ} = 10T_{INS}$ and $T_{RES} = \frac{T_{INS}}{2}$.

(g) Primitive checking loop (degenerate form of polling) time is T_{chk} where $T_{chk} = \frac{T_{INS}}{3} = 2T_{clk}$.

(h) The result of the execution of instruction at 33, is used up in instruction $K > 34$ (and the processor would not be able to proceed further without executing instruction K).

Note: In figure above, read T_{CPU} as T_{INS}.

Compute

(a) Total clock cycles required in Cases A through C (for the whole program to complete). Use the notation $T_{prptl}, T_{poll}, T_{intrrpt}$ for the total clock cycles in each case

(b) Which of the schemes (A through C) above is most efficient, in terms of execution time (the lesser, the better) for the whole program to complete. Support your arguments with quantitative results. [Show the inequalities involving $T_{prptl}, T_{poll}, T_{intrrpt}$].

(c) Compute the value (or range of values) of N_1 and N_2 such that the CPU need not wait for results to execute statement number K.

C → K − 33 T_INS ≥ N1 T_INS

A → CPU won't wait

B → Time taken to 33 to K ≥ T_I/O

⇒ (K − 33) T_INS ≥ N1 T_INS

$T_{INS} = 6T_{CLK}$ $T_{I/O} = 6N_1T_{CLK}$ $T_{REQ} = 60T_{CLK}$ $T_{RES} = 3T_{CLK}$ $T_{CHK} = 2T_{CLK}$

A: No interrupt / polling T_{REQ} T_{CHK} $T_{I/O}$ T_{RES}

for I/O operation:

$$T_{33} = 60T_{CLK} + 6N_1T_{CLK} + 3T_{CLK}$$

Total: $99 \times T_{INS} + T_{33} = (657 + 6N_1)T_{CLK}$

B: polling: $T_{33} = (63 + 6N_1)T_{CLK}$

$$K_{poll} = \left\lfloor \frac{N_1 T_{INS}}{N_2 T_{INS} + T_{CHK}} \right\rfloor$$

No of ins executed concurrently along w 33rd ins

$$= K_{poll} N_2$$

$$T_{tot} = 32 \times 6T_{CLK} + T_{33} + (67 - K_{poll}N_2)\,6T_{CLK} = \left(657 + \frac{6N_1}{3N_2 + 1}\right)T_{CLK}$$

C: Interrupt:

$$32 \times 6T_{CLK} + T_{33} + (67 - N_1)\,6T_{CLK} = 657\,T_{CLK}$$